# ASSIGNMENT 4 – CO4

# TASK 2: FROM REQUIREMENT TO EVENT-DRIVEN DESIGN

Hospital Outpatient Appointment Desk

**Course:** Programming in Java (CSA09)
**Course Outcome:** CO4 – Event-driven GUI design
**Task Weightage:** 8 Marks

Design report covering event modelling, component hierarchy, handler pseudocode, event tracing and complexity analysis.

# 1. Requirement Analysis

The outpatient appointment desk is used by a receptionist to book a patient's appointment. The receptionist enters a patient ID, chooses a department, selects a doctor belonging to that department, selects an available time slot and submits the booking. The application must show a clear confirmation or error. If a booking operation is in progress, an attempt to close the window must first ask the receptionist to confirm whether the operation should be cancelled/closed.

**Primary user:** Receptionist. **Core data:** patient ID, department, doctor, time slot and booking status. **Core rule:** the doctor choices must correspond to the selected department.

**Design assumptions:** Department and doctor choices are available when the application starts; doctor data is grouped by department in memory. The task does not prescribe a database schema or a particular booking service, so the design treats the booking operation as a service call. A booking is considered in progress from submission until the service returns a success or failure result.

# 2.1 Task 2.1 – Model the Events

| Event source | Event object | Listener interface | Action taken |
|---|---|---|---|
| Department selector (JComboBox / Choice) | ItemEvent (or ActionEvent for JComboBox) | ItemListener (or ActionListener) | Read selected department; refresh doctor selector; clear stale doctor selection. |
| Doctor selector (JComboBox / Choice) | ItemEvent (or ActionEvent) | ItemListener (or ActionListener) | Record selected doctor; optionally refresh dependent availability. |
| Time-slot selector | ItemEvent (or ActionEvent) | ItemListener (or ActionListener) | Record selected slot and enable booking when required fields are valid. |
| Book button | ActionEvent | ActionListener | Validate fields; mark booking in progress; invoke booking service; display confirmation/error. |
| Window close control | WindowEvent | WindowListener / WindowAdapter | If booking is in progress, ask for confirmation; close only when safe/confirmed. |
| Confirmation dialog buttons | ActionEvent | ActionListener (or JOptionPane result) | Continue closing or return to the appointment window. |

Note: In Swing, JComboBox commonly uses ActionListener for selection changes; ItemListener is also suitable when selection-state changes are specifically required. In AWT, Choice commonly uses ItemListener. The event object and listener should match the actual component selected for implementation.

# 2.2 Task 2.2 – Design the Component Hierarchy

Recommended implementation: Swing. A JFrame is the top-level window. A BorderLayout divides the main content into a title/header area, a central form and a status area. GridBagLayout (or GridLayout for a simpler fixed form) aligns labels and controls in the form.

```
JFrame  AppointmentFrame  [BorderLayout]
|
+-- JPanel  headerPanel  [FlowLayout / BorderLayout.NORTH]
|   +-- JLabel  titleLabel  ("Outpatient Appointment Desk")
|
+-- JPanel  formPanel  [GridBagLayout / BorderLayout.CENTER]
|   +-- JLabel      patientIdLabel
|   +-- JTextField  patientIdField
|   +-- JLabel      departmentLabel
|   +-- JComboBox   departmentCombo
|   +-- JLabel      doctorLabel
|   +-- JComboBox   doctorCombo
|   +-- JLabel      timeSlotLabel
|   +-- JComboBox   timeSlotCombo
|   +-- JButton     bookButton
|   +-- JButton     clearButton  (optional convenience)
|
+-- JPanel  statusPanel  [FlowLayout / BorderLayout.SOUTH]
    +-- JLabel  statusLabel
```

A modal confirmation dialog (JOptionPane) is displayed only when the user attempts to close the frame while a booking is in progress. A separate confirmation/error dialog or status label communicates the booking result.

## 2.3 Task 2.3 – Design the Handler Logic

### (a) Refreshing the doctor list when the department changes

```
EVENT: Department selection changes
INPUT: selectedDepartment
PRECONDITION: department selection is not null

BEGIN
    selectedDepartment <- departmentCombo.getSelectedItem()
    doctorCombo.removeAllItems()

    IF selectedDepartment is null THEN
        doctorCombo.setEnabled(FALSE)
        RETURN
    END IF

    doctorList <- doctorMap.get(selectedDepartment)

    IF doctorList is empty THEN
        doctorCombo.addItem("No doctors available")
        doctorCombo.setEnabled(FALSE)
        statusLabel.setText("No doctors available for this department")
    ELSE
        FOR EACH doctor IN doctorList
            doctorCombo.addItem(doctor)
        END FOR
        doctorCombo.setEnabled(TRUE)
        doctorCombo.setSelectedIndex(0)
        statusLabel.setText("Select a doctor and time slot")
    END IF
END
```

### (b) Validating and booking the appointment

```
EVENT: Book button is pressed
BEGIN
    IF bookingInProgress = TRUE THEN
        RETURN
    END IF

    patientId <- patientIdField.getText().trim()
    department <- departmentCombo.getSelectedItem()
    doctor <- doctorCombo.getSelectedItem()
    timeSlot <- timeSlotCombo.getSelectedItem()

    IF patientId is empty THEN
        show error "Enter patient ID"
        focus patientIdField
        RETURN
    END IF

    IF department is null OR doctor is null OR timeSlot is null THEN
        show error "Select department, doctor and time slot"
        RETURN
    END IF

    IF doctor does not belong to department THEN
        show error "Selected doctor does not match department"
        RETURN
    END IF

    bookingInProgress <- TRUE
    disable Book button
    statusLabel <- "Booking appointment..."

    START background booking operation
        result <- AppointmentService.book(patientId, department,
                                          doctor, timeSlot)

        ON UI EVENT THREAD:
            bookingInProgress <- FALSE
            enable Book button
```

```
            IF result is successful THEN
                show confirmation with booking reference
                statusLabel <- "Appointment booked successfully"
            ELSE
                show clear error message
                statusLabel <- "Booking failed"
            END IF
        END BACKGROUND OPERATION
    END
```

The background-operation step is a design recommendation to keep a potentially slow service/database request off Swing's Event Dispatch Thread. In a Swing implementation, SwingWorker is an appropriate mechanism; UI updates must be made on the Event Dispatch Thread. The assignment asks for handler pseudocode, not a complete implementation.

## 2.4 Task 2.4 – Trace the Event Sequence

| Step | Method/event | What happens |
|---|---|---|
| 1 | main() | SwingUtilities.invokeLater(...) schedules GUI creation on the Event Dispatch Thread. |
| 2 | AppointmentFrame constructor | Creates frame, panels, labels, selectors and buttons; registers listeners. |
| 3 | setVisible(true) | Displays the window; layout and painting are performed by Swing. |
| 4 | departmentCombo selection event | ActionListener/ItemListener runs; doctor list is refreshed. |
| 5 | Book button ActionEvent | Booking handler validates fields and starts the booking operation. |
| 6 | Booking result callback / SwingWorker.done() | Updates status and shows success/error; clears bookingInProgress. |
| 7 | Window close requested | Window event is delivered; windowClosing handler checks bookingInProgress. |
| 8 | Confirmation choice | If booking is in progress, show confirm dialog; cancel keeps window open; confirm follows the chosen safe-close policy. |
| 9 | dispose() | Releases the frame's native resources and closes the window. |

**Applet life-cycle comparison:** An applet has container-managed life-cycle callbacks such as init(), start(), paint(), stop() and destroy(). A Swing JFrame is not an applet and does not use that life cycle. A desktop window is created by application code, shown with setVisible(true), responds to registered event listeners, and is closed/disposed through window handling. Painting is managed by Swing's repaint mechanism rather than by an applet's paint() life-cycle sequence.

# 3. Alternative Concept and Complexity Analysis

Alternative: fetch the doctor list from the database every time the receptionist changes the department, instead of loading all departments and doctors once into Map>.

| Aspect | Preloaded Map approach | Database fetch on every change |
|---|---|---|
| Initial work | Load/group doctor data once; build map. | Little/no initial doctor-list loading. |
| Department change time | Expected O(1) map lookup, plus O(r) to populate the UI with r doctors for the selected department. | Database round-trip/query cost plus O(r) to populate the UI; latency depends on database/network and query execution. |
| Repeated changes | No repeated database round trips; fast local lookup. | Each change repeats query and network/database overhead. |
| Memory | O(d + m) map/list structure for d departments and m doctor records (excluding object/string overhead). | Lower application-side cached doctor-data memory, approximately O(r) for the currently displayed list, but database still stores the data. |
| Responsiveness | Fast selection response after initial load; initial loading should be done asynchronously if large. | Can freeze the UI if query runs on the Event Dispatch Thread; use SwingWorker/background work. |

| Aspect | Preloaded Map approach | Database fetch on every change |
|---|---|---|
| Freshness | May become stale until refreshed/reloaded. | Can reflect more recent database changes, depending on transaction/consistency. |

**Complexity interpretation:** Let d be the number of departments, m the total number of doctor records and r the number of doctors in the currently selected department. Building the map requires O(d + m) time and O(d + m) additional memory. Once built, retrieving one department's list is expected O(1); adding its r doctors to the combo box costs O(r). A database-backed change has query execution and round-trip latency in addition to O(r) UI population; its exact Big-O depends on indexes, schema and query plan, so it should not be falsely represented as a universal constant-time operation.

**Conclusion:** For a relatively stable doctor directory, preloading into a map is preferable because department changes are fast and do not repeatedly access the database. If doctor availability changes frequently or the directory is very large, a database fetch can be preferable for freshness and reduced client memory, but it must run in a background worker.

## 4. Verification Checklist

• Changing department refreshes the doctor list and removes doctors from the previous department.

• Changing to a department with no doctors displays a clear message and disables booking.

• Booking with an empty patient ID is rejected with a useful error.

• Booking without department, doctor or time slot is rejected.

• A valid selection starts one booking operation and prevents duplicate clicks while it is in progress.

• Success and failure both update the status and restore the Book button.

• Closing while idle closes normally; closing while booking is in progress asks for confirmation.

• All Swing component updates occur on the Event Dispatch Thread.

## 5. Submission Note

This report answers the design and analysis requirements of Task 2. The task does not request Java source-code implementation. Keep this report in the Task02 folder of the required GitHub repository. Add your own screenshots only if you create a prototype or implementation beyond the task's stated evidence requirements. Do not label a design sketch as an executed program screenshot.